PLAN DE PRUEBAS

Sistema bajo prueba: Demoblaze, tienda en línea de demostración

VERSIÓN 1.0

Joshua Mendez

Edinson Pedroza

Construcción de Software y Pruebas Grupo A 2026-2

Profesora: Claudia Martínez

HOJA RESUMEN DE MODIFICACIONES

Versión	Fecha	Cambios respecto de la versión anterior	Preparado por	Aprobado por
1.0	11 de octubre de 2026	Versión inicial	Joshua Mendez Edinson Pedroza	Claudia Martínez

ÍNDICE

Hoja resumen de modificaciones . . . 2
Sistema bajo prueba y nota preliminar . . . 4
1. Alcance . . . 4
1.1 Dentro del alcance . . . 4
1.2 Fuera del alcance . . . 4
2. Criterios de entrada y salida (DoR y DoD) . . . 5
2.1 Definition of Ready: criterios de entrada . . . 5
2.2 Definition of Done: criterios de salida . . . 6
3. Estrategia de pruebas . . . 6
3.1 Niveles y tipos . . . 6
3.2 Técnicas de diseño de casos . . . 6
3.3 Tabla de decisión de la compra . . . 7
3.4 Criterios de referencia . . . 7
3.5 Pruebas asociadas al cambio . . . 8
3.6 Priorización . . . 8
3.7 Ejecución manual, y qué se automatizaría . . . 8
3.8 Hallazgo de la exploración preliminar . . . 8
3.9 Integración entre sistema y producto: resultados . . . 8
3.10 Rendimiento, carga y estrés . . . 9
3.11 Seguridad . . . 11
3.12 Compatibilidad de navegadores . . . 11
3.13 Cronograma de ejecución . . . 14
4. Entregables . . . 16
5. Dependencias . . . 16
6. Entorno de pruebas . . . 17
7. Gestión de riesgos . . . 17
7.1 Clasificación de severidad de defectos . . . 17
7.2 Restricciones asumidas, que no son riesgos . . . 18
7.3 Riesgos identificados . . . 18
8. Roles y responsabilidades . . . 18
Referencias . . . 19

Sistema bajo prueba y nota preliminar

Este plan cubre **Demoblaze** (`https://www.demoblaze.com`), una tienda en línea de demostración publicada por BlazeMeter, que es uno de los sitios propuestos por la docente para el ejercicio.

Trabajamos en grupo: Joshua Mendez y Edinson Pedroza. El reparto de funciones entre los dos está en la sección de roles.

Algo importante antes de empezar: el sistema no es nuestro. No tengo el código fuente, ni entro al servidor, ni veo la base de datos o los logs, y tampoco puedo pedir que arreglen un defecto. Eso cambia para qué sirve este plan. No estamos certificando un producto antes de sacarlo a producción, porque ya está en producción. Lo que hacemos es medir la calidad que se puede ver desde afuera y dejar anotado lo que encontremos. Por eso hay varios criterios típicos de un plan de pruebas que tuvimos que adaptar, y lo avisamos en cada sección donde pasa.

El documento sigue las ocho secciones que pide el taller. Para armarlas nos apoyamos en Katalon (s.f.), y para los nombres de niveles y tipos en Martínez (2026a, 2026b) e ISTQB (2018).

1. Alcance

1.1 Dentro del alcance

Recorrimos el sitio entero y revisamos sus elementos para listar todo lo que el usuario puede hacer. La tabla cubre todas las funciones que cambian algo en el sistema. Lo que es solo presentación y no cambia nada, como el carrusel de la portada, el pie de página y el video de About us, lo dejamos junto en la última fila y de eso solo revisamos que cargue bien.

1.2 Fuera del alcance

- **Pruebas de componente.** Requieren acceso al código fuente (Martínez, 2026a) y no lo hay.
- **Pruebas de integración entre componentes.** Esas sí necesitan el código y quedan fuera. La integración entre sistema y producto, en cambio, sí entra: está en el apartado 3.9 con los resultados.
- **Pruebas de denegación de servicio.** Inundar el sitio con tráfico afectaría a las demás personas que lo usan, así que quedan fuera. El resto de pruebas de seguridad sí entra, y están en el apartado 3.11 con los resultados.
- **El reproductor de video de la ventana About us**, más allá de comprobar que la ventana abre y el reproductor carga.
- **Persistencia de datos a largo plazo.** El sitio de demostración se reinicia cada tanto y nadie garantiza que los datos se queden. Probar que un usuario creado hoy siga ahí la otra semana no diría nada útil.
- **Automatización de la ejecución.** Esta iteración es manual. En la sección 3 indico qué se automatizaría y por qué no se hace ahora.

Tabla 1: Funcionalidad incluida en el alcance.

Módulo	Qué se prueba
Catálogo	Listado de productos en la página principal, filtrado por las tres categorías (Phones, Laptops, Monitors), paginación, y consistencia entre el precio del listado y el del detalle.
Detalle de producto	Carga de la ficha por identificador, datos mostrados (nombre, precio, descripción, imagen) y acción de agregar al carrito.
Carrito	Agregar uno y varios productos, eliminar, cálculo del total, persistencia del contenido al navegar entre páginas.
Registro (Sign up)	Alta de usuario con datos válidos, usuario duplicado, campos vacíos y límites de longitud.
Autenticación (Log in)	Ingreso con credenciales válidas e inválidas, campos vacíos, y cierre de sesión.
Orden de compra	Formulario de compra, con el total precargado y sus seis campos editables (nombre, país, ciudad, tarjeta, mes, año): validaciones, confirmación y contenido del comprobante.
Contacto	Envío del formulario con datos válidos e inválidos y comportamiento con campos vacíos.
Interfaz y accesibilidad	Asociación de etiquetas con campos en los formularios, navegación por teclado y comportamiento responsivo básico.
Elementos de presentación	Carga sin error del carrusel de la portada, del pie de página y del reproductor de la ventana About us. No se verifica su contenido.
Rendimiento	Métricas web esenciales de la página principal contra los umbrales publicados por Google, medidas sobre usuarios reales en escritorio y en móvil. Diseño de las pruebas de carga y de estrés con su perfil, herramienta y umbrales.
Integración entre sistema y producto	Contrato entre la interfaz y el servicio HTTP que consume: códigos de respuesta, forma de los datos, consistencia entre respuestas y manejo de errores en la interfaz.
Seguridad	Cabeceras de seguridad de la respuesta, manejo de la contraseña en el login, enumeración de usuarios, inyección en el login y política CORS de la API. No incluye denegación de servicio.
Compatibilidad de navegadores	Que la tienda cargue y se vea bien en los tres motores de navegador que existen (Blink, Gecko y WebKit), en escritorio y en móvil. La matriz por riesgo y lo cubierto van en el apartado 3.12.

2. Criterios de entrada y salida (DoR y DoD)

Uso el DoR para las condiciones que se tienen que cumplir antes de empezar a probar, y el DoD para las que permiten decir que las pruebas ya terminaron. Partí de las listas de entrada y salida de Katalon (s.f.) y las ajustamos a un sistema que no controlo.

2.1 Definition of Ready: criterios de entrada

E1. Este plan está revisado y aprobado.

E2. Los casos de prueba están escritos, con resultado esperado explícito, y revisados antes de ejecutarse.

E3. Una prueba rápida de sanidad sale bien: carga la página principal, abre la ficha de un producto y abre el carrito. Si esto falla, ni empiezo, porque los resultados no dirían nada del software sino de que el sitio está caído.

E4. El navegador de referencia está instalado y su versión queda registrada.

E5. Los datos de prueba están definidos: usuario y contraseña ficticios, número de tarjeta ficticio y valores de borde para cada campo.

E6. Existe una plantilla de registro de ejecución y otra de reporte de defectos.

2.2 Definition of Done: criterios de salida

S1. El 100 % de los casos de prioridad alta está ejecutado, y al menos el 80 % del total.

S2. Al menos el 90 % de los casos ejecutados resulta aprobado. Tomo este umbral del documento de ejemplo entregado con el taller, que fija una aceptación del 90 % sobre el total ejecutado.

S3. Todo defecto encontrado está registrado con pasos de reproducción, evidencia gráfica y severidad asignada según la clasificación de la sección 7.

S4. Los casos que fallaron por una causa ajena al software, como una caída del sitio, están reejecutados o marcados como no concluyentes con su justificación.

S5. El informe de resumen de pruebas está entregado, con las métricas de la sección 4.

3. Estrategia de pruebas

Antes de entrar, una aclaración de nombres. Katalon (s.f.) dedica todo su artículo a separar el plan de pruebas de la estrategia de pruebas. El plan es de un proyecto puntual; la estrategia es algo más general, de toda una organización. Así que lo que va en esta sección no es una estrategia de empresa sino cómo pensamos probar este sitio en concreto.

3.1 Niveles y tipos

Como no tengo el código, las pruebas de componente no van. El grueso del trabajo se hace en el nivel de sistema, revisando el producto de punta a punta como lo vería un usuario (Martínez, 2026a). La integración entre sistema y producto sí se puede probar por caja negra, y la desarrollo en el apartado 3.9.

Casi todos los casos son funcionales. Hay una parte menor de usabilidad, accesibilidad y rendimiento, que es no funcional (Martínez, 2026b).

3.2 Técnicas de diseño de casos

- **Partición de equivalencias** en los campos de texto de los cuatro formularios, separando entradas válidas, inválidas y vacías.
- **Análisis de valores límite** en la longitud de usuario y contraseña, y en los campos de mes y año del formulario de compra, que son los únicos dominios ordenados con un borde definido.
- **Transición de estados** para el ciclo del carrito: vacío, con un producto, con varios, tras agregar dos veces el mismo, tras eliminar y tras completar la compra.
- **Tabla de decisión** para la compra, desarrollada en el apartado siguiente.

No todos los casos salen de una técnica de diseño. Algunos son revisiones de consistencia, como comparar el comprobante con el total del carrito, y otros son inspecciones, como mirar si las etiquetas de los formularios apuntan al campo que toca.

3.3 Tabla de decisión de la compra

La compra depende de tres condiciones de sí o no, así que salen ocho combinaciones. La tabla las muestra todas, marca cuáles voy a probar y dice por qué dejo las otras por fuera.

Tabla 2: Tabla de decisión del formulario de compra.

#	Sesión	Carrito	Campos	Resultado esperado	Caso
1	Sí	Con productos	Completos	Compra exitosa con comprobante	CP-COM-01
2	Sí	Con productos	Incompletos	Rechazo por campos obligatorios	CP-COM-04
3	Sí	Vacío	Completos	Rechazo por carrito vacío	CP-COM-02
4	No	Con productos	Completos	Se exige iniciar sesión	CP-COM-03
5	Sí	Vacío	Incompletos	Rechazo	Descartada
6	No	Con productos	Incompletos	Rechazo	Descartada
7	No	Vacío	Completos	Rechazo	Descartada
8	No	Vacío	Incompletos	Rechazo	Descartada

Las combinaciones 5 a 8 las descartamos porque cada una junta dos o tres motivos de rechazo que ya revisamos por separado en la 2, la 3 y la 4. No nos dirían nada nuevo, y si el sistema fallara sería más difícil saber cuál de los motivos lo causó.

3.4 Criterios de referencia

El resultado esperado de cada caso se fija contra estándares públicos, no contra un criterio propio: comparo el sitio con lo que se le exige a cualquier software.

Modelo de calidad. La norma ISO/IEC 25010 lista ocho características de calidad: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad. De esas ocho puedo mirar cinco desde afuera: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad y fiabilidad. Mantenibilidad y portabilidad necesitan el código, y seguridad queda fuera por lo que ya expliqué en el alcance. Cada módulo de la Tabla 1 lo pruebo contra la característica que le toca.

Accesibilidad. Aquí el criterio son las WCAG 2.1 del W3C. El hallazgo del apartado 3.8, donde las etiquetas de los formularios no apuntan a sus campos, rompe dos criterios de nivel A: el 1.3.1, información y relaciones, y el 4.1.2, nombre, función y valor.

Rendimiento. Google publica umbrales concretos para las métricas web esenciales y son los que uso en el apartado 3.10: Largest Contentful Paint de máximo 2,5 segundos, Interaction to Next Paint de máximo 200 milisegundos y Cumulative Layout Shift de máximo 0,10.

Convenciones del dominio. Cuando no hay estándar que aplique, el criterio es lo que cualquiera espera de una tienda en línea: que no deje comprar con el carrito vacío, que el total dé igual a la suma de lo que muestra, que el precio del listado sea el mismo del detalle y que un error de login no deje adivinar si el usuario existe. Son expectativas comunes a cualquier comercio electrónico.

En la suite, cada caso indica en una columna cuál de estos cuatro criterios se le aplica.

3.5 Pruebas asociadas al cambio

Yo no despliego nada, pero el sitio puede cambiar de un día para otro sin avisar. Por eso dejo armada desde ya una regresión mínima: el flujo de compra completo más el registro y el inicio de sesión. Si noto que algo cambió durante las pruebas, la vuelvo a correr. La prueba de confirmación queda definida por si llegan a arreglar algún defecto.

3.6 Priorización

Ordeno el trabajo por riesgo y no por lo que sea más cómodo. Primero el flujo de compra, que es para lo que existe una tienda y donde una falla pega más fuerte. Después login y registro, porque de ellos depende poder comprar. Al final catálogo, contacto y los temas de interfaz.

3.7 Ejecución manual, y qué se automatizaría

Esta vuelta la hacemos manual, porque con esta cantidad de casos todavía no se justifica automatizar. Si el ejercicio siguiera, automatizaría la regresión del apartado 3.5, que es la que se repite. Un detalle a tener en cuenta: el sitio confirma varias acciones con diálogos de JavaScript, y esos diálogos frenan la automatización si la herramienta no está preparada para manejarlos. Eso habría que resolverlo antes.

3.8 Hallazgo de la exploración preliminar

Mientras reconocíamos el sitio, revisando el árbol de accesibilidad nos dimos cuenta de que en los formularios *Place order* y *Contact* las etiquetas no están bien asociadas a sus campos. El campo que va después de *Name* se anuncia como *Total*, y el que va después de *Contact Name* se anuncia como *Contact Email*. Un lector de pantalla leería un nombre que no es el del campo. Por eso incluí la accesibilidad en el alcance.

3.9 Integración entre sistema y producto: resultados

La interfaz de Demoblaze no calcula nada por su cuenta: todo se lo pide a un servicio HTTP aparte, en `api.demoblaze.com`. Esa unión sí la puedo probar por caja negra, mirando el tráfico del navegador y llamando al servicio directo. Corrí cuatro casos y encontré tres problemas.

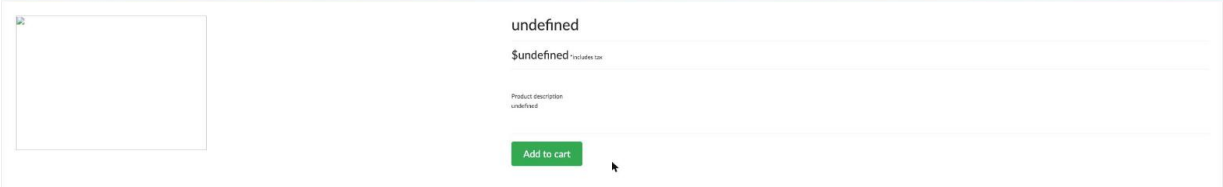


Figura 1: Página de producto con un identificador que no existe. El servicio responde 200 y la interfaz muestra *undefined*.

El CP-ISP-04 es el más serio de los tres, y es porque junta dos errores. El servicio miente con el código de estado, y la interfaz le cree sin mirar el cuerpo de la respuesta. Un cliente que valide solo por el código, que es lo normal, tomaría un producto inexistente como si existiera. También lo probamos desde la interfaz: al darle a agregar al carrito sobre ese producto fantasma, el sitio responde “Product added” y el carrito queda vacío. Me confirmó una acción que en realidad no pasó.

Tabla 3: Resultados de las pruebas de integración entre la interfaz y su servicio.

Caso	Llamada	Resultado observado	Estado
CP-ISP-01	`GET /entries`	Responde 200 en 0,44 s con 9 productos. La portada muestra esos mismos 9.	Aprobado
CP-ISP-02	`POST /bycat`	Responde 200 con preflight CORS. Devuelve 7 teléfonos, 6 portátiles y 2 monitores, o sea 15 productos. Pero `/entries` solo devuelve 9. Los dos servicios pagina distinto y la portada nunca muestra 6 de los 15 productos hasta que uno pasa de página.	Fallido
CP-ISP-03	`POST /view`	Responde 200 y el título y el precio coinciden con lo que trae `/entries` para el mismo id.	Aprobado
CP-ISP-04	`POST /view` con id inexistente	Responde **200 OK** con el cuerpo `{"errorMessage":"Not found."}`. Debería responder 404. La interfaz no revisa ese caso y pinta *undefined* en título, precio y descripción, con el botón de agregar al carrito funcionando.	Fallido

Ya que estaba mirando el tráfico, vi otra cosa: el video de la ventana About us se descarga en todas las páginas, incluida la ficha de producto, aunque la ventana esté cerrada y el video no se vea nunca. Son segmentos `.ts` que se bajan sin que nadie los pida, y eso ayuda a explicar los tiempos del apartado siguiente.

3.10 Rendimiento, carga y estrés

Lo que sí medí. Medí la página principal con una herramienta pública de análisis de rendimiento web, que muestra las métricas tomadas de usuarios reales de Chrome en los últimos 28 días. La Figura 1 es la captura de esa medición.



Figura 2: Métricas web esenciales de Demoblaze, medidas sobre usuarios reales el 14 de septiembre de 2026.

En escritorio la herramienta marca el resultado global como **desaprobada**, y por una sola métrica: el desplazamiento acumulado llega a 0,12 y el umbral es 0,10. O sea, el contenido se mueve mientras la

Tabla 4: Resultados de rendimiento contra los umbrales publicados por Google.

Métrica	Medido	Umbral bueno	Estado
Largest Contentful Paint (LCP)	1,4 s	hasta 2,5 s	Cumple
Interaction to Next Paint (INP)	45 ms	hasta 200 ms	Cumple
Cumulative Layout Shift (CLS)	0,12	hasta 0,10	**No cumple**
First Contentful Paint (FCP)	1,3 s	hasta 1,8 s	Cumple
Time to First Byte (TTFB)	0,7 s	hasta 0,8 s	Cumple

página carga. Cuadra con lo que se ve en el sitio: el carrusel de la portada aparece después del resto y empuja los productos hacia abajo.

En móvil está bastante peor, y es la vista con la que la mayoría entra a una tienda. Ahí no falla una métrica sino tres.

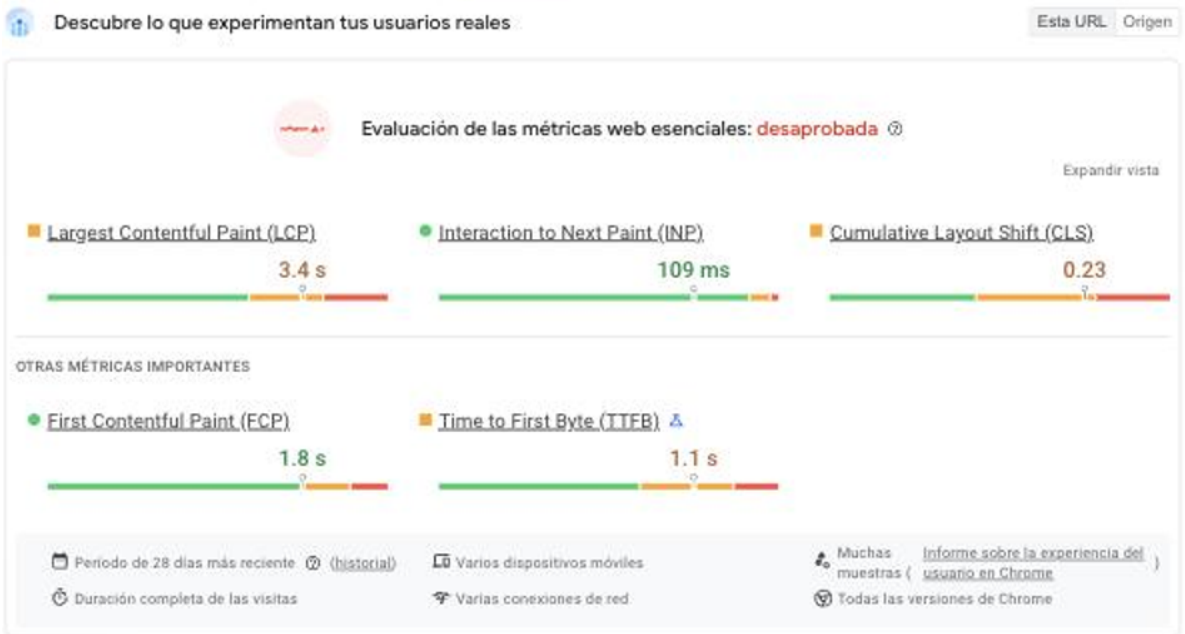


Figura 3: Las mismas métricas medidas en móvil. Fallan LCP, CLS y TTFB.

El desplazamiento se duplica en móvil, de 0,12 a 0,23, y el tiempo hasta el primer byte se pasa del umbral. Con lo que vi en el tráfico del apartado anterior, donde el video se baja en todas las páginas sin que nadie lo pida, tengo una explicación probable, aunque habría que confirmarla midiendo con el video apagado. Lo anoto como defecto medio en móvil y leve en escritorio.

Carga y estrés. Estas dos las dejo diseñadas pero sin correr, por lo mismo que las de seguridad: meterle tráfico artificial a un servidor ajeno y sin permiso del dueño le arruinaría el sitio a los demás. El diseño de las dos pruebas es este:

- **Herramienta:** una herramienta de pruebas de carga por línea de comandos, con reporte de percentiles.
- **Guion:** el flujo de compra completo, que es el crítico: abrir la portada, entrar a un producto, agregarlo al carrito y terminar la orden.

Tabla 5: Comparación entre escritorio y móvil.

Métrica	Escritorio	Móvil	Umbral	En móvil
Largest Contentful Paint	1,4 s	3,4 s	hasta 2,5 s	No cumple
Interaction to Next Paint	45 ms	109 ms	hasta 200 ms	Cumple
Cumulative Layout Shift	0,12	0,23	hasta 0,10	No cumple
First Contentful Paint	1,3 s	1,8 s	hasta 1,8 s	Justo en el límite
Time to First Byte	0,7 s	1,1 s	hasta 0,8 s	No cumple

- **Prueba de carga:** subir hasta 50 usuarios concurrentes en 5 minutos y sostenerlos 15 minutos. Umbral: percentil 95 del tiempo de respuesta por debajo de 3 segundos y menos del 1 % de errores.
- **Prueba de estrés:** sumar 25 usuarios cada 3 minutos hasta que la tasa de error pase del 5 % o el percentil 95 pase de los 10 segundos. Lo que espero sacar: el punto de saturación anotado y evidencia de que el sistema baja de a poco en vez de caerse de golpe.
- **Métricas a recoger:** tiempo de respuesta por percentiles, transacciones por segundo, tasa de error y cuánto tarda en recuperarse al quitarle la carga.

3.11 Seguridad

Corrí pruebas de seguridad suficientes para resolver el taller pero no para afectar el servicio: un sondeo a nivel de la API y de las cabeceras, con seis casos. Cinco fallaron.

La más grave es la configuración CORS de CP-SEG-06. Como refleja cualquier origen y a la vez permite credenciales, deja que un sitio de terceros le haga peticiones a la API en nombre de un usuario con la sesión abierta. Qué tanto daño haga depende de cómo viaje la sesión, pero reflejar un origen cualquiera con las credenciales activadas es un error de configuración que hay que arreglar. Detrás vienen la falta de cabeceras y la enumeración de usuarios, esta última ya confirmada por los dos lados en CP-SEG-05. La contraseña en base64 es la más leve, porque HTTPS la protege en tránsito. La inyección en el login no logró saltarse la autenticación, aunque la respuesta rara de CP-SEG-04 queda anotada para mirarla después.

3.12 Compatibilidad de navegadores

Probar todos los navegadores, versiones y dispositivos no es viable, y tampoco es lo que se hace en la práctica: se arma una matriz por riesgo sobre lo que de verdad usa la gente. Se simplifica porque en el mundo solo hay tres motores de navegador: Blink (Chrome, Edge, Brave, Opera), Gecko (Firefox, Tor) y WebKit (Safari). Si la tienda se ve bien en los tres, en escritorio y en móvil, quedan cubiertos casi todos los usuarios.

Tabla 7: Matriz de compatibilidad por motor y plataforma.

Tabla 6: Resultados de las pruebas de seguridad.

Caso	Qué se probó	Resultado observado	Estado
CP-SEG-01	Cabeceras de seguridad de la respuesta	La página no devuelve ninguna cabecera de seguridad: falta Content-Security-Policy, Strict-Transport-Security, X-Frame-Options y X-Content-Type-Options. Eso la deja expuesta a clickjacking y a que el navegador adivine el tipo de contenido.	Fallido
CP-SEG-02	Manejo de la contraseña en el login	El servicio de login recibe la contraseña como base64. Por ejemplo, `dGVzdDEyMw==` es simplemente `test123` codificado. Viaja sobre HTTPS, así que va cifrada en tránsito, pero base64 es reversible y no reemplaza a un hash.	Fallido
CP-SEG-03	Enumeración de usuarios (mensaje)	Al intentar entrar con un usuario que no existe, el servicio responde con el texto exacto `User does not exist.`, distinto del que da una contraseña incorrecta. Ese mensaje deja averiguar qué nombres de usuario están registrados.	Fallido
CP-SEG-04	Bypass de autenticación con inyección en el login	Tres cadenas de inyección en el usuario no lograron entrar: ninguna devolvió un token de sesión. La autenticación no se saltó. Eso sí, las tres respondieron `Wrong password.` en vez de `User does not exist.`, o sea que el servicio trató la cadena de inyección como un usuario que existe. No es un bypass, pero es una respuesta rara que valdría la pena mirar con más calma.	Aprobado
CP-SEG-05	Barrido de enumeración	Los usuarios `admin`, `test` y `administrator` responden `Wrong password.`, mientras que un nombre al azar responde `User does not exist.` Queda confirmado que las cuentas existentes se pueden distinguir de las que no, una por una.	Fallido
CP-SEG-06	Política CORS de la API	Ante una petición desde el origen inventado `https://sitio-atacante.example`, la API responde `Access-Control-Allow-Origin` con ese mismo origen y además `Access-Control-Allow-Credentials: true`. O sea, acepta peticiones con credenciales desde cualquier sitio web.	Fallido

Motor	Navegador	Plataforma	Estado	Resultado
Blink	Chrome 138	Escritorio, Win 10	Ejecutado	La tienda carga completa y bien maquetada.
Gecko	Tor Browser 14.5	Escritorio, Win 10	Ejecutado	Funciona. El menú superior se parte en dos líneas (DEF-14, leve).
Blink	Chrome Android	Móvil	Ejecutado, datos de campo	El apartado 3.10 usa datos reales de usuarios de Chrome en Android: la tienda funciona, con reparos de rendimiento (DEF-07).
WebKit	Safari, macOS	Escritorio	Ejecutado	La tienda se ve idéntica a Chrome: menú en una línea, carrusel con imagen y grilla de tres columnas (Figura 6).
WebKit	Safari, iOS	Móvil	Ejecutado	En un iPhone real. La tienda carga completa y funcional: menú, categorías, los ocho productos con imagen y el pie de página (Figura 7).
Blink	Tienda en iPad, 1024x768	Tablet, layout	Ejecutado	El layout de escritorio se mantiene intacto: carrusel, grilla de tres columnas y menú en una línea.
Blink	Tienda en iPhone, 375x667	Móvil, layout	Ejecutado	Las categorías y los productos se apilan bien en una columna, pero el menú no colapsa a un botón y el carrusel se ve vacío (DEF-15).

Para escritorio usé un servicio que abre navegadores reales en la nube. Blink y Gecko los ejecutamos ahí (Figura 4). La única diferencia entre los dos es cosmética: en Tor el menú se parte en dos líneas porque el navegador trae su propia fuente y las letras quedan más anchas. El sitio se usa igual en los dos.

Para los distintos tamaños de pantalla usé una herramienta que carga la tienda en un marco del tamaño

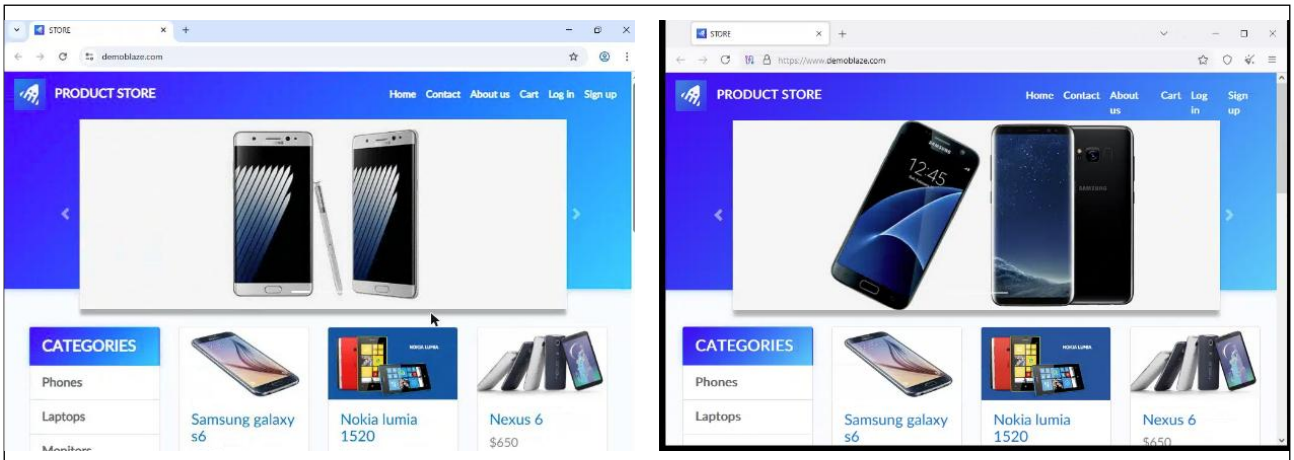


Figura 4: Demoblaze en Chrome, a la izquierda, y en Tor Browser, a la derecha. En el motor de Firefox los enlaces del menú superior se parten en dos líneas.

de cada dispositivo, desde laptop hasta teléfono. Pude enmarcarla justamente porque el sitio no manda la cabecera X-Frame-Options (el DEF-10 de seguridad), así que un tercero lo puede meter en un iframe. Probé varios tamaños y el resultado es claro: de tablet para arriba la tienda se ve igual que en escritorio, pero en ancho de teléfono el diseño se degrada. La Figura 5 lo muestra.

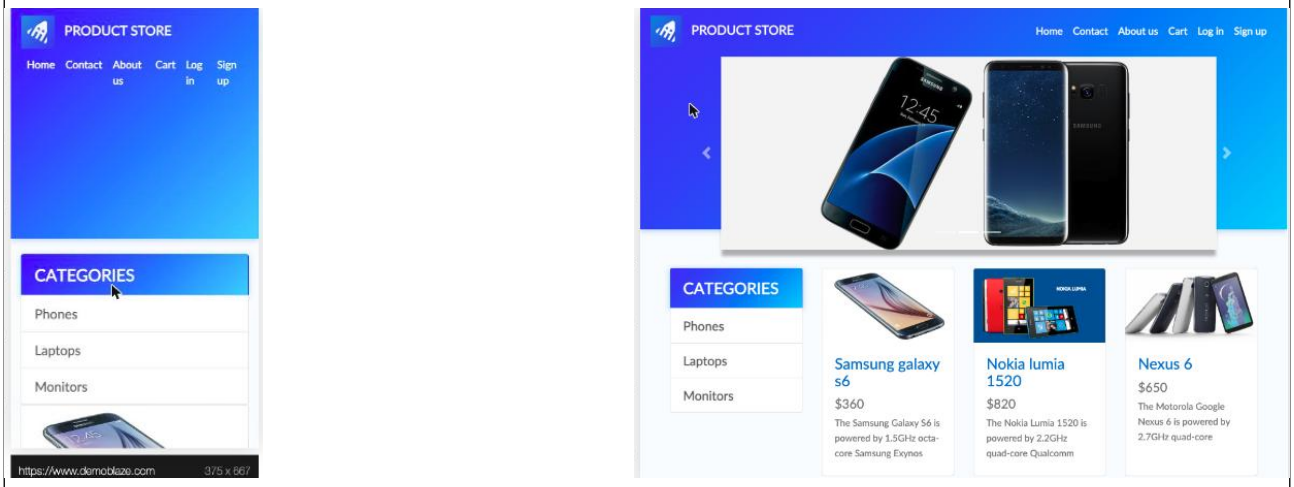


Figura 5: Demoblaze a distintos tamaños de pantalla. A la izquierda, en iPhone (375 px): el menú no colapsa a un botón y el carrusel queda vacío. A la derecha, en iPad (1024 px): el diseño de escritorio se mantiene completo.

En el teléfono la tienda sigue siendo usable, pero el menú superior se amontona en dos filas en vez de recogerse en un botón hamburguesa, que es lo que uno esperaría de un sitio pensado para móvil, y el carrusel de la portada aparece vacío. Lo anoto como DEF-15, de severidad media, porque el móvil es de donde llega buena parte de los usuarios de una tienda.

El motor WebKit en escritorio lo cerré con Safari sobre macOS. La tienda se ve igual que en los otros dos motores, sin diferencias de maquetación (Figura 6). Con eso, los tres motores del mundo quedan probados en escritorio, y en móvil quedan cubiertos Blink por layout y por datos de campo (Chrome en Android).

La última celda, Safari en iOS, se cerró con un iPhone real: la tienda carga completa y se puede usar (Figura 7). Ese render viene de un guardado como PDF desde el teléfono, que usa ancho de impresión y por eso muestra el menú en una línea y no el layout exacto de 375 px; los detalles finos de esa medida son

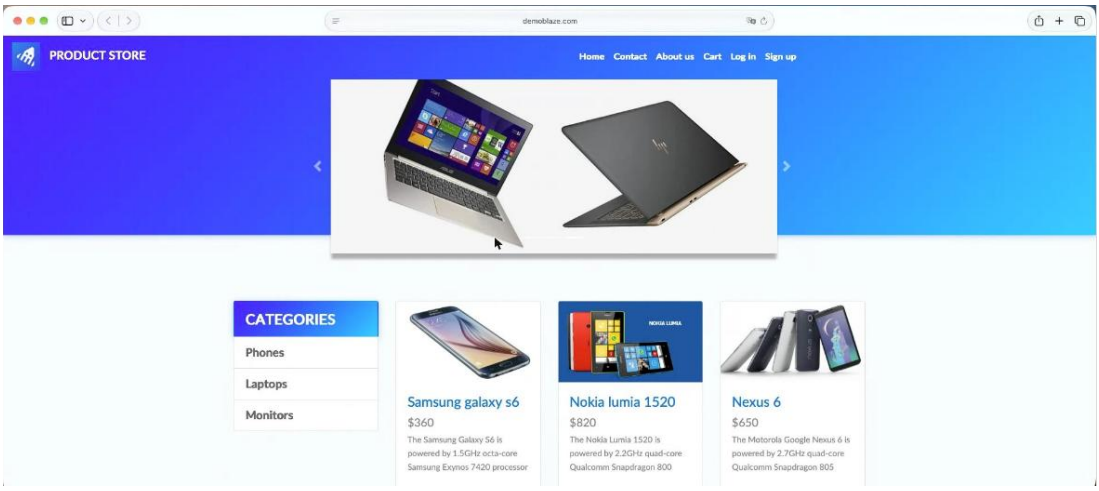


Figura 6: Demoblaze en Safari (motor WebKit) sobre macOS. El render es idéntico al de Chrome.

los que ya recoge la Figura 5. Aun así, confirma lo que importa: WebKit sobre iOS muestra el sitio entero y funcional.

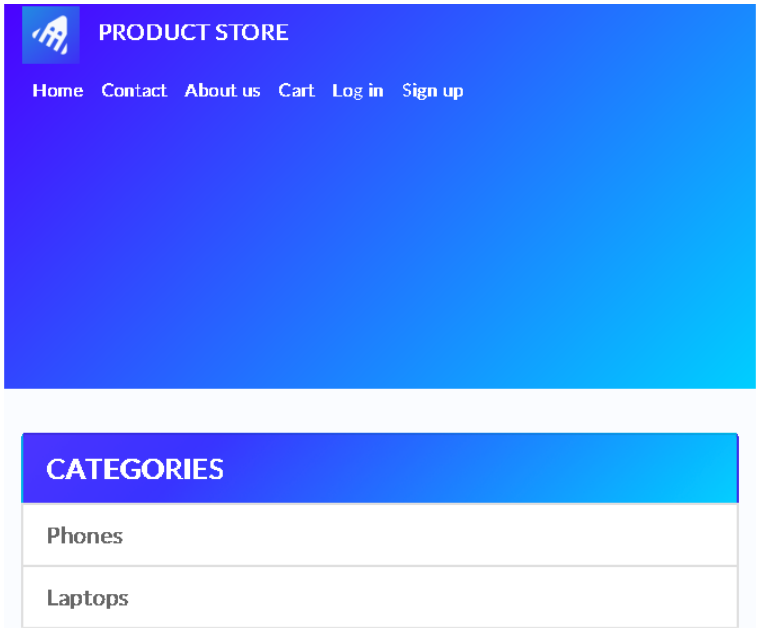


Figura 7: Demoblaze en Safari sobre un iPhone real. La tienda carga completa: menú, categorías y productos con sus imágenes.

Con esto, los tres motores del mundo quedan probados en escritorio y en móvil. El resto de combinaciones (versiones antiguas, navegadores marginales) se prioriza según a quién afecten.

3.13 Cronograma de ejecución

El cronograma ordena la ejecución. Dejé los bloques cortos a propósito, para que si el sitio se cae no se pierda mucho trabajo.

Si se acaba el tiempo, corto al final de B1. Con los casos de prioridad alta ya corridos se cumple el

Tabla 8: Bloques de ejecución previstos.

Bloque	Actividad	Salida
B0, 30 min	Verificación de los criterios de entrada y prueba de sanidad.	Ambiente confirmado o ejecución abortada.
B1, 2 h	Los 22 casos de prioridad alta: compra, carrito, autenticación, registro, integración, rendimiento y seguridad.	Criterio S1 cubierto.
B2, 2 h	Los 27 casos de prioridad media.	Cobertura por encima del 90 %.
B3, 45 min	Los 5 casos de prioridad baja y reejecución de los que quedaron bloqueados. Dos de ellos, carga y estrés, quedan marcados como no ejecutables en este entorno.	Suite completa.
B4, 1 h	Consolidación de defectos e informe de resumen.	Criterios de salida evaluados.

criterio S1, y el informe se puede entregar anotando la cobertura parcial como desviación.

4. Entregables

Esta tabla lista lo que sale del proceso de pruebas, con qué lleva cada cosa y en qué formato. Tres de los seis ya están hechos y van junto con este plan.

Tabla 9: Entregables del proceso de pruebas.

Entregable	Contenido y formato
Plan de pruebas	Este documento. Formato PDF.
Suite de casos de prueba	Hoja de cálculo con una fila por caso: identificador, módulo, prioridad, técnica aplicada, precondiciones, pasos, datos de prueba, resultado esperado y criterio de referencia. Esta última columna materializa lo descrito en el apartado 3.4: cada caso declara contra cuál de los cuatro criterios se juzga el resultado.
Registro de ejecución	Identificador del caso, fecha y hora, versión del navegador, resultado (aprobado, fallido, bloqueado, no concluyente) y referencia a la evidencia.
Reporte de defectos	Uno por defecto: identificador, título, pasos de reproducción, resultado obtenido contra el esperado, severidad, evidencia y módulo afectado.
Evidencias	Capturas de pantalla asociadas a cada caso fallido y a cada defecto.
Informe de resumen	Casos diseñados y ejecutados, tasa de aprobación, defectos por severidad y por módulo, desviaciones respecto del plan y conclusión sobre el cumplimiento de los criterios de salida.

Para que se entienda el formato, así queda un caso de la suite:

Tabla 10: Ejemplo de un caso de prueba en el formato definido.

Identificador	CP-COM-04
Módulo	Orden de compra
Prioridad	Alta
Técnica	Tabla de decisión
Precondición	Carrito con al menos un producto y ventana de compra abierta.
Pasos	1. Dejar vacíos los campos Nombre y Tarjeta. 2. Completar País y Ciudad con valores válidos. 3. Pulsar Purchase.
Resultado esperado	El sistema impide la compra y señala los campos obligatorios sin completar. No se genera comprobante ni se vacía el carrito.

5. Dependencias

D1. **Que el sitio esté arriba.** Es la dependencia más importante. Demoblaze es un servicio gratuito de un tercero, sin acuerdo de nivel de servicio y sin ningún compromiso de estar disponible. Si se cae, no puedo hacer nada.

D2. **Estabilidad de los datos de demostración.** Los casos del catálogo se apoyan en los productos y categorías existentes. Si el proveedor los cambia, hay que actualizar los resultados esperados.

D3. **Conectividad a internet** desde el equipo de pruebas.

D4. **Navegador de referencia** en la versión declarada en la sección 6.

D5. **Que el equipo pueda trabajar.** Al ser solo dos personas, la baja de cualquiera reduce a la mitad la capacidad y recarga al otro.

Hay una diferencia con un proyecto normal. Katalon (s.f.) dice que las dependencias típicas son módulos o servicios que el equipo de desarrollo todavía no termina, y recomienda dejar módulos simulados para esos casos. Aquí eso no pasa: el sistema ya está completo y en producción desde antes de que empezáramos. Todas nuestras dependencias son externas y ninguna la puedo negociar. Eso hace la planeación más predecible, pero también significa que no puedo hacer nada si alguna falla.

6. Entorno de pruebas

Tabla 11: Configuración del entorno de pruebas.

Elemento	Configuración
Sistema bajo prueba	`https://www.demoblaze.com`, entorno de producción del proveedor.
Navegador de referencia	Google Chrome. La versión exacta se registra el día de ejecución y se anota en el registro, porque el navegador puede actualizarse solo entre sesiones.
Sistema operativo	macOS.
Resolución de pantalla	1920 por 1080, con una verificación adicional a 1366 por 768 para el comportamiento responsivo básico.
Datos de prueba	Usuario y contraseña ficticios creados para el ejercicio, número de tarjeta ficticio, y valores de borde por campo. No se emplean datos personales reales.
Herramientas	Navegador con sus herramientas de desarrollo para inspeccionar la interfaz, y captura de pantalla para la evidencia.
Accesos no disponibles	Código fuente, servidor, base de datos, registros de aplicación y consola de administración.

La diferencia grande con un plan normal es que no hay un ambiente de pruebas aparte. Pruebo directamente sobre la producción del proveedor, la misma que está usando cualquier otra persona en ese momento. De ahí salen dos cosas que afectan todo el plan: no puedo hacer pruebas destructivas ni de carga, y los datos pueden cambiar de una sesión a otra porque alguien más los tocó. Las dos quedan anotadas en la sección siguiente, donde separo lo que son restricciones de lo que son riesgos de verdad.

7. Gestión de riesgos

7.1 Clasificación de severidad de defectos

Uso la clasificación del documento de ejemplo que venía con el taller, ajustada a este sitio:

- **Crítico**: impide completar el flujo de compra, muestra información errónea de precios o totales, o expone funciones que deberían requerir sesión iniciada.
- **Medio**: falla una función secundaria, o una validación no impide una entrada claramente inválida sin llegar a bloquear la compra.
- **Leve**: defectos de presentación, textos, coherencia visual o comportamientos distintos ante situaciones equivalentes.

7.2 Restricciones asumidas, que no son riesgos

Hay tres condiciones de este ejercicio que ya sabía antes de escribir el plan, así que su probabilidad es del 100 por ciento. No son riesgos, porque un riesgo es algo incierto que se puede gestionar. Son restricciones y se asumen. Las separo para no llenar la tabla de riesgos con cosas que no lo son.

C1. **No hay ambiente de pruebas separado.** Se trabaja sobre la producción compartida del proveedor. De ahí la prohibición de pruebas de carga, estrés e intrusivas, y el uso exclusivo de datos ficticios.

C2. **Los defectos encontrados no se pueden corregir.** El objetivo se redefine hacia documentar y evaluar impacto en lugar de certificar, según el apartado 2.3.

C3. **La ejecución se hace en una sola ventana de trabajo.** De ahí la priorización del apartado 3.6 y el criterio de salida S1, que exige el 100 % de los casos de prioridad alta y no del total.

7.3 Riesgos identificados

Tabla 12: Riesgos del proceso de pruebas y su mitigación.

Cód.	Riesgo	Probab.	Impacto	Mitigación
R1	El sitio queda fuera de servicio durante la ejecución.	Media	Alto	Ejecutar en los bloques cortos del apartado 3.13 y capturar la evidencia de cada caso en el momento, sin dejarla para el final.
R2	El proveedor cambia el sitio o reinicia los datos a mitad del trabajo.	Media	Medio	Diseñar casos independientes entre sí, cada uno con su propia precondición. Reejecutar la suite de regresión si se detecta un cambio.
R3	El mismo equipo que diseña los casos los ejecuta, con riesgo de sesgo de confirmación.	Media	Medio	Que una persona diseñe cada caso y la otra lo revise y ejecute, para que el diseño y la ejecución no recaigan en la misma persona.
R4	El criterio de referencia elegido no corresponde al caso y se reporta como defecto un comportamiento correcto.	Media	Medio	Declarar el criterio en la columna respectiva de la suite, de modo que la decisión quede auditable y se pueda revisar si alguien discrepa.
R5	La evidencia capturada no permite reproducir el defecto después.	Media	Medio	Capturar la pantalla completa con la URL visible y anotar los datos exactos de entrada en el registro de ejecución.
R6	El sitio usa diálogos de JavaScript para confirmar acciones, lo que bloquea la automatización.	Media	Bajo	Ejecución manual en esta iteración. Evaluar el manejo de diálogos antes de automatizar.

R3 es el mismo problema de independencia que aparece cuando un equipo prueba lo que él mismo construyó. Con solo dos personas no se puede eliminar, pero sí atenuar haciendo que quien diseña un caso no sea quien lo ejecuta, como indica la mitigación de la tabla.

8. Roles y responsabilidades

Los roles se organizan por función, indicando quién asume cada una. Al ser un equipo de dos, cada integrante concentra varias responsabilidades, pero el diseño de los casos y su ejecución se asignan a personas distintas a propósito, por lo que se explica debajo de la tabla.

Tabla 13: Roles, responsabilidades y asignación.

Rol	Responsabilidad	Asignado a
Líder de pruebas	Definir alcance, estrategia y criterios de entrada y salida. Aprobar el cierre de la actividad.	Joshua Mendez
Diseñador de casos	Derivar los casos de prueba aplicando las técnicas de la sección 3 y documentar el resultado esperado de cada uno.	Joshua Mendez
Ejecutor de pruebas	Ejecutar los casos, registrar resultados y recolectar evidencia.	Edinson Pedroza
Gestor de defectos	Registrar y clasificar los defectos, evaluar su impacto y redactar el informe de resumen.	Edinson Pedroza
Aprobador	Revisar el plan y el informe final, y dar por recibida la entrega.	Claudia Martínez, docente del curso

El reparto separa a propósito el diseño de la ejecución: Joshua Mendez diseña los casos y Edinson Pedroza los ejecuta. Así atacamos el riesgo R3 de la sección anterior, porque el sesgo de confirmación pega más fuerte cuando quien escribió un caso es quien lo ejecuta —tiende a correrlo esperando que salga como lo pensó—. Con dos personas no se elimina del todo (ambos redactamos el plan y revisamos el trabajo), pero sí se reduce al no concentrar diseño y ejecución en la misma mano.

Referencias

Google. (2024). *Web Vitals: umbrales de las métricas web esenciales*. https://web.dev/articles/vitals

International Organization for Standardization. (2011). *ISO/IEC 25010:2011. Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models*. ISO.

International Software Testing Qualifications Board. (2018). *Certified Tester Foundation Level syllabus* (cap. 2). ISTQB.

Katalon. (s.f.). *Test plan vs. test strategy: key distinctions and components*. `https://katalon.com/resources-center/blog/test-plan`

Martínez, C. (2026a). *Niveles de pruebas* [Diapositivas]. Construcción de Software y Pruebas, Pontificia Universidad Javeriana.

Martínez, C. (2026b). *Tipos de prueba* [Diapositivas]. Construcción de Software y Pruebas, Pontificia Universidad Javeriana.

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C. https://www.w3.org/TR/WCAG21/